CO1 ASSIGNMENT 1

C Programming – Complete Solutions with Code and Expected Output

Based on the uploaded assignment: Grama Jyothi Rural Electricity Co-operative billing system.

Important note: The outputs below are sample outputs for a typical system where `unsigned short` is 2 bytes, `int` is 4 bytes, and `long long` is 8 bytes. Exact sizes can vary by compiler and platform. Compile with `gcc -std=c11 -Wall -Wextra`.

Main design assumptions

• Meter register is a 16-bit unsigned counter with range 0–65535; one rollover is allowed between two readings.

• The first 100 units are free; units 101–200 cost ■2.25 each; units above 200 cost ■4.50 each.

• Fixed charge is ■50 per bill. Money is stored as integer paise (■1 = 100 paise).

• A reading pair is valid if it is non-negative and either current ≥ previous or a genuine rollover occurred.

• The simple rollover formula assumes no more than one complete rollover between readings.

Overall data flow

Read household ID, category and meter readings → validate input → calculate units (including rollover) → calculate slab-wise energy charge → add fixed charge → print rupees and paise.

TASK 1 – Understand the Problem and Identify the Data

Inputs, outputs and justification

Data item	Meaning / reason	Suggested representation
Household number	Identifies each consumer; should be positive.	unsigned int / unsigned long
Category	Small code such as domestic or commercial.	enum or char
Previous/current reading	Meter register values 0–65535.	meter_t (unsigned short)
Units consumed	Difference between readings; may exceed 65535 only if multiple rollovers are allowed.	unsigned int for one rollover
Energy charge	Tariff-based money; exact to paise.	long long paise
Fixed charge	■50 = 5000 paise.	constant long long
Total bill	Fixed + energy charge; never negative.	long long paise

Constants, rules and assumptions

• Constants: register maximum 65535, slab boundaries 100 and 200 units, rates 0/225/450 paise per unit, fixed charge 5000 paise.

• A correct bill must satisfy: units ≥ 0; energy charge ≥ 0; total = fixed charge + energy charge; money has no fractional paise.

• Use integer paise so the result is exact. Do not use a floating-point value as the stored money amount.

• Text readings such as “01530” require O(d) time to parse and compare d digits; numeric readings take O(1) arithmetic time. Text needs about d bytes plus terminator; a 16-bit reading needs 2 bytes.

Text representation alternative

With strings, validate that every character is a digit, convert using strtoul(), then compare numeric values. Conversion and comparison take O(d) time per reading. Integer representation uses constant-time comparison/subtraction and fixed storage. Strings preserve leading zeroes but use more memory.

TASK 2 – Choose the Data Types

Recommended data types

Value	Type	Reason
Meter reading	unsigned short	16-bit register; typical size 2 bytes; range 0–65535.
Units consumed	unsigned int	Non-negative; enough for one rollover.
Household ID	unsigned long	Supports large consumer counts.
Tariff / paise totals	long long	Exact integer money and large aggregate totals.
Category	enum	Named fixed set of categories.
Input text	char array	For reading and validating text input.

Complete TODO: rollover-safe consumption

```
#include <stdio.h>
#include <limits.h>
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (unsigned short)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\\n", consumption(1200, 1530));
    printf("Consumption: %u units\\n", consumption(65400, 120));
    return 0;
}
```

Expected output

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

The subtraction is performed with unsigned arithmetic and wraps modulo 65536. For rollover: (65536 – 65400) + 120 = 136 + 120 = 256. It does not become negative.

Float versus integer paise

A float stores an approximation of many decimal fractions. Repeatedly adding a value such as ■0.10 can accumulate a small binary rounding error. Store each bill as an integer number of paise and add integers; the result is exact until the integer type overflows. Addition is O(1) in both approaches; a float is commonly 4 bytes and long long commonly 8 bytes.

```
/* Demonstration idea */
float ftotal = 0.0f;
long long ptotal = 0;
for (int i = 0; i < 4000; ++i) {
    ftotal += 0.10f;  /* approximate */
    ptotal += 10;     /* exact: 10 paise */
}
```

TASK 3 – Define the Constants

Why use macros?

#define is suitable for compile-time tariff constants and has no storage of its own. A const variable has a type, scope and debugger visibility. An enum is best for named integer choices such as consumer categories. Use uppercase names for constants.

Completed billing function

```
#include <stdio.h>
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS)
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    if (units > SLAB1_UNITS) {
        int slab2 = (units < SLAB2_UNITS) ? units - SLAB1_UNITS
                                          : SLAB2_UNITS - SLAB1_UNITS;
        paise += (long)slab2 * RATE2_PAISE;
    }
    /* First slab costs zero, so no addition is needed. */
    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = (int)(sizeof tests / sizeof tests[0]);
    for (int i = 0; i < n; ++i) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Expected output

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

If RATE2_PAISE changes from 225 to 230, every unit in the second slab uses the new rate because the value is defined once. The program must be recompiled after changing a macro. A const variable is type-checked and visible in a debugger; a macro is text substitution. Running time is effectively unchanged.

TASK 4 – Use Macros Safely

Safe macros

```
#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

Examples and pitfalls

• SQUARE(a + 1) expands safely to ((a + 1) * (a + 1)). Without parentheses, x*x can expand with incorrect operator precedence.

• MAX(i++, j) may evaluate i++ more than once because the chosen macro expression can use an argument in both the comparison and result. Avoid side effects in macro arguments.

• A macro is expanded by the preprocessor. Run: gcc -E program.c -o program.i and inspect the expanded source.

• A function or static inline function is safer because arguments are evaluated once and types are checked.

Inline-function alternative

```
static inline long long square_ll(long long x)
{
    return x * x;
}

static inline long long max_ll(long long a, long long b)
{
    return (a > b) ? a : b;
}
```

Both macro expansion and inline calls are generally O(1) for these operations. A macro used k times duplicates its expanded expression at each use and may increase code size; an inline function may be expanded by the compiler or kept as a call. Prefer inline functions for type safety and side-effect safety.

TASK 5 – Organise with Header Files and Conditional Compilation

billing.h

```
#ifndef BILLING_H
#define BILLING_H

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000LL

typedef unsigned short meter_t;
unsigned int consumption(meter_t previous, meter_t current);
long long compute_bill(int units);

#endif
```

billing.c

```
#include "billing.h"

unsigned int consumption(meter_t previous, meter_t current)
{
    return (unsigned short)(current - previous);
}

long long compute_bill(int units)
{
    long long p = FIXED_CHARGE_PAISE;
    if (units > 200) p += (long long)(units - 200) * RATE3_PAISE;
    if (units > 100) {
        int n = (units < 200) ? units - 100 : 100;
        p += (long long)n * RATE2_PAISE;
    }
#ifdef DEBUG
    printf("DEBUG: units=%d, bill_paise=%lld\\n", units, p);
#endif
    return p;
}
```

Add to billing.c when using printf. The main file includes billing.h and calls the functions. Include guards prevent the same header from being processed more than once. Use #include for standard/system headers and #include "billing.h" for your own header.

Compile

```
gcc -std=c11 -Wall -Wextra main.c billing.c -o billing
gcc -std=c11 -Wall -Wextra -DDEBUG main.c billing.c -o billing_debug
```

With #ifdef DEBUG, debug statements are removed from a normal build. A run-time if (debug) keeps the test and debug code in the executable, and checks the flag whenever execution reaches it. The run-time check is O(1) per check; compile-time selection avoids that check.

TASK 6 – Read and Print Data Correctly

Format specifiers

• %d – int; %u – unsigned int; %ld – long; %lld – long long.

• %f – float in printf; %lf – double in scanf (for printf, %f prints a double).

• %c – char; %s – character string.

Validated input and money printing

```
#include <stdio.h>

int main(void)
{
    int previous, current;
    long long bill_paise;

    printf("Enter previous and current readings: ");
    if (scanf("%d %d", &previous, &current) != 2) {
        printf("Invalid input\\n");
        return 1;
    }
    if (previous < 0 || current < 0) {
        printf("Reading cannot be negative\\n");
        return 1;
    }

    bill_paise = 16250; /* example: Rs 162.50 */
    printf("Bill = Rs %lld.%02lld\\n",
           bill_paise / 100, bill_paise % 100);
    return 0;
}
```

Sample output

```
Enter previous and current readings: 1200 1530
Bill = Rs 162.50
```

Always check scanf's return value. If it returns fewer items than expected, the input was not successfully converted. Using %d for a long is a format mismatch and causes undefined behaviour. With fgets(), read a whole line into a bounded buffer, then use strtol()/sscanf() to validate it. Processing a line of L characters takes O(L) time and O(L) buffer space; it is easier to discard a bad line and recover than with direct scanf.

TASK 7 – Control Type Conversion and Precision

Three surprising expressions

```
printf("%d\\n", 5 / 2);       /* 2, integer division */
printf("%.1f\\n", 5 / 2.0);   /* 2.5, floating division */
printf("%d\\n", (int)3.9);    /* 3, cast truncates */
```

Explanation and safe rules

• 5 / 2 uses integer operands, so the fractional part is discarded. Use 5.0 / 2 or cast one operand.

• Casting a large long long to int can lose information or produce an implementation-defined result when it is outside int range. Check the range before casting.

• When a fractional charge must be converted to paise, define a rounding rule, for example round to the nearest paise using a suitable rounding function. Prefer tariffs already expressed in paise.

• Integer promotions convert small integer types before arithmetic; usual arithmetic conversions choose a common type. Cast before multiplication when the result may exceed int.

Float alternative

If all bills are accumulated in float and rounded only once, each addition can introduce a small representation error. The total error can grow with n additions. Integer paise gives exact additions as long as the accumulator does not overflow. Typical storage: float 4 bytes; long long 8 bytes. Both additions are O(1).

TASK 8 – Trace the Compilation Process

GCC commands and outputs

```
gcc -E billing.c -o billing.i   # preprocessing
gcc -S billing.c -o billing.s   # assembly generation
gcc -c billing.c -o billing.o   # object file
gcc main.o billing.o -o billing # linking
```

• Preprocessing: expands #include and #define; removes comments and selects conditional-compilation branches.

• Compilation: translates C into assembly; assembly stage creates machine-code object files.

• Linking: combines object files and libraries into an executable.

• Compile-time error: missing semicolon or undeclared variable.

• Link-time error: calling a function whose definition is not linked (undefined reference).

• Run-time error: invalid input handling or an arithmetic fault while the program runs.

• Use -Wall -Wextra to identify format mismatches, unused variables and other suspicious code; fix warnings rather than ignoring them.

Multi-file build alternative

For input.c, tariff.c and report.c, compile each source file separately into an object file, then link them with a shared header. If one .c file changes, only that file normally needs recompilation before relinking. In a single-file build, the whole source is recompiled. Separate object files use additional disk space, but make large projects easier to maintain.

TASK 9 – Implement and Test the Billing Program

Complete billing program

```
#include <stdio.h>
#include <limits.h>

#define RATE2 225LL
#define RATE3 450LL
#define FIXED 5000LL

typedef unsigned short meter_t;

unsigned int units_used(meter_t oldr, meter_t newr)
{
    return (unsigned short)(newr - oldr);
}

long long bill_for(int units)
{
    long long total = FIXED;
    if (units > 100) {
        int second = (units < 200) ? units - 100 : 100;
        total += (long long)second * RATE2;
    }
    if (units > 200)
        total += (long long)(units - 200) * RATE3;
    return total;
}

int main(void)
{
    meter_t oldr = 65400, newr = 120;
    unsigned int u = units_used(oldr, newr);
    long long p = bill_for((int)u);

    printf("Rollover units = %u\\n", u);
    printf("Bill = Rs %lld.%02lld\\n", p / 100, p % 100);
    return 0;
}
```

Expected output

```
Rollover units = 256
Bill = Rs 527.00
```

For 256 units: fixed ■50 + (100 × ■2.25) + (56 × ■4.50) = ■50 + ■225 + ■252 = ■527.

Test table

Test	Expected result
0 units	■50.00
100 units	■50.00
150 units	■162.50
200 units	■275.00
350 units	■950.00
65400 → 120	256 units; ■527.00
Letters / negative reading	Reject as invalid
Current < previous without valid rollover	Reject or flag as suspicious

A one-rollover formula cannot distinguish a true rollover from an invalid backwards reading in every case. A real system should also use timestamps and a maximum plausible consumption check. For s slabs stored in arrays, a loop computes the bill in O(s) time and O(s) configuration space; fixed if-statements are constant time but less flexible.

TASK 10 – Compare Expectations with Execution

Aggregate total example

For a simple scale comparison, assume every household has the same ■50.00 bill (5,000 paise). This is an illustrative baseline, not a measured real-world tariff distribution.

Consumers	Total in paise	Total in rupees
4,000	20,000,000	■200,000.00
60,000	300,000,000	■3,000,000.00
800,000	4,000,000,000	■40,000,000.00

Type comparison

• A 32-bit signed int commonly has maximum 2,147,483,647; 4,000,000,000 paise exceeds it, so int is insufficient for this illustrative district total.

• A 64-bit long long can safely hold 4,000,000,000 paise. Check LLONG_MAX from limits.h on the actual system.

• On many 64-bit Linux systems, long is 8 bytes; on some other systems it is 4 bytes. Use sizeof and limits.h rather than assuming.

• double has a much wider range than integer types but represents decimal values approximately. It may lose exact paise precision for sufficiently large or repeated fractional sums.

• Integer paise and double additions are both O(1); long long commonly uses 8 bytes and double commonly uses 8 bytes.

Commands to measure

```
printf("int=%zu long=%zu long long=%zu double=%zu\\n",
       sizeof(int), sizeof(long), sizeof(long long), sizeof(double));
printf("INT_MAX=%d LLONG_MAX=%lld\\n", INT_MAX, LLONG_MAX);
```

Use the actual output from your own compiler in the final submission. Do not claim the sample sizes are guaranteed by every C implementation.

TASK 11 – Recommend and Defend the Final Design

Final recommendation

• Use unsigned short for the 16-bit meter register, unsigned int for one-rollover consumption, and long long for paise totals.

• Keep tariff rates and slab limits in one clearly named location. For a small fixed tariff, macros or const variables are acceptable; use const variables where type checking and debugger visibility are useful.

• For frequent tariff revisions, load and validate a configuration file at program start instead of editing macros and recompiling.

• Validate every reading, household identifier and tariff value. Log invalid records and retain an audit trail.

• The design scales to 800,000 households if aggregate money uses a sufficiently wide integer type and the program checks overflow.

Configuration-file alternative and complexity

If t tariff values are read from a configuration file, startup parsing takes O(t) time and O(t) storage. This avoids recompiling and redistributing the program for each tariff revision. Validate the file (non-negative rates, increasing slab limits, valid time periods) and use a safe default or stop if the file is invalid. Recompiling has a larger deployment cost but keeps configuration simple for a very small fixed tariff.

TASK 12 – Re-evaluate and Adapt the Design – Smart Meter Challenge

Decision: MODIFY the original design

• The meter now sends watt-hour readings every 15 minutes: 35,040 records per house per year. The program must process timestamped intervals, not just one monthly difference.

• A 16-bit unsigned short is no longer suitable for yearly accumulated watt-hours. Use uint64_t (from stdint.h) for cumulative energy and long long / int64_t for money in paise.

• Store each interval's timestamp and watt-hours. Convert watt-hours to kilowatt-hours only for reporting; retain integer watt-hours internally.

• The evening period 6 PM–10 PM needs time-zone-aware timestamp classification. Apply the 20% surcharge only to energy consumed during those intervals.

• Tariff values must come from a configuration file or database at startup, because the board wants revisions without recompiling.

Suggested interval data and calculation

```
#include <stdint.h>

typedef struct {
    int year, month, day;
    int hour, minute;
    uint64_t watt_hours;
} MeterInterval;

/* Example: base rate is paise per kWh.
   For an interval, calculate energy charge with integer arithmetic
   and a documented rounding rule. */
```

For an interval in the 6 PM–10 PM period, multiply the base charge by 120 and divide by 100, rounding once to the nearest paise. To avoid premature rounding, retain a numerator in a wider integer type and round at the final billing boundary. Check multiplication for overflow.

Complexity and final architecture

• Processing N interval records takes O(N) time; storing all yearly records takes O(N) space. A streaming design can process records one at a time and keep only running totals, reducing working memory to O(1) per household (apart from audit storage).

• A district with 800,000 homes produces 28,032,000,000 interval records per year (800,000 × 35,040), so use batch processing, indexing by household/time and durable storage.

• Keep meter ingestion, tariff lookup, billing calculation and report generation in separate modules. Test boundaries at 17:59, 18:00, 21:59 and 22:00, daylight-saving/time-zone rules where applicable, missing records and duplicate records.

• The original integer-paise principle should be retained; the meter data model and tariff system must be modified.

Conclusion

The safest design uses integer energy and money representations, centralized validated tariff data, strict input checking, and tests for slab boundaries and meter rollover. For smart meters, process timestamped watt-hour intervals and apply the time-of-day tariff before accumulating the final bill.